

Objectives

- Working with Strings, Arrays, and Object
- Using Comparison and Logical Operators
- Writing Conditional Statements
- Working with Loops

Strings, Arrays, and Object

Strings

Strings are used to store text like names, messages, etc. In JavaScript, we use either double quotes `" "`, single quotes `' '`, or backticks `` `` for template literals.

```
let name = "Alice";  
let greeting = 'Hello, world!';
```

String Operator

The `+` operator is the primary operator for string operations in JavaScript. When used with strings, it performs concatenation (combining them together):

```
let greeting = "Hello" + " " + "World"; // "Hello World"
```

String Indexing

We can treat strings as arrays of characters, accessing individual characters using zero-based indexing

```
const word = "hello";  
word[0]; // 'h' (first character)  
word[4]; // 'o' (last character)
```

String Methods

Case Conversion

We can use case conversion methods to modify the letter casing of strings in JavaScript.

```
'hello'.toUpperCase(); // 'HELLO'  
'HELLO'.toLowerCase(); // 'hello'
```

Searching & Checking

JavaScript provides methods to search and check strings, allowing us to:

1. Verify if a substring exists

```
'JavaScript'.includes('Script'); // true
```

2. Find character/word positions

```
'hello'.indexOf('e'); // 1 (first occurrence)  
'hello'.lastIndexOf('l'); // 3 (last occurrence)
```

3. Check starting/ending patterns

```
'file.txt'.startsWith('file'); // true  
'image.png'.endsWith('.png'); // true
```

Extraction & Slicing

JavaScript provides several methods to extract portions of strings:

1. **slice()**: Extracts a section between two indices:

```
'JavaScript'.slice(0, 4); // 'Java'  
'JavaScript'.slice(4); // 'Script' (to end)  
'JavaScript'.slice(-6); // 'Script' (from end)
```

2. **substring()**: Similar to slice() but doesn't accept negative indices:

```
'JavaScript'.substring(0, 4); // 'Java'
```

3. **substr()**: Uses start index and length:

```
'JavaScript'.substr(4, 3); // 'Scr'
```

4. **charAt()**: Gets a specific character:

```
'Hello'.charAt(1); // 'e'
```

Modification

JavaScript provides methods to create modified versions of strings (remembering that strings themselves are immutable).

1. **Trimming Whitespace**

- `trim()` : Removes whitespace from both ends
- `trimStart()` : Removes only leading whitespace
- `trimEnd()` : Removes only trailing whitespace

```
' hello '.trim();      // 'hello'
' hello '.trimStart(); // 'hello '
' hello '.trimEnd();   // '  hello'
```

2. **Replacing Text**

- `replace()` : Replaces the first matching pattern
- `replaceAll()` : Replaces all matching patterns

```
'apple orange apple'.replace('apple', 'banana'); // 'banana orange apple'
'apple orange apple'.replaceAll('apple', 'banana'); // 'banana orange banana'
```

3. **Repeating Strings**

- `repeat()` Creates repeated copies:

```
'ha'.repeat(3); // 'hahaha'
```

4. **Padding Strings**

- `padStart()` Adds characters to the **beginning** (left side) of a string until it reaches the specified length.
- `padEnd()` Adds characters to the **end** (right side) of a string until it reaches the specified length.

```
'5'.padStart(3, '0'); // '005'
'5'.padEnd(3, '!');  // '5!!!'
```

Splitting & Combining

1. **Splitting Strings:** `split(separator)` divide a string into an array:

```
'a,b,c'.split(',');      // ['a', 'b', 'c']
'hello'.split('');       // ['h', 'e', 'l', 'l', 'o']
```

2. **Combining Arrays:** `join(separator)` merge an array into a string:

```
['a', 'b', 'c'].join('-'); // 'a-b-c'
['2023', '10', '05'].join('/'); // '2023/10/05'
```

Arrays

We can do much more than simply access arrays and their elements using array methods, we can:

- **Add** or **remove** elements
- **Search** for specific items
- **Transform** array contents
- **Sort** and **reorganize** data
- **Combine** or **split** arrays

Getting Array Length

We can determine the number of elements in an array using the `length` property:

```
let fruits = ['apple', 'banana', 'orange'];
console.log(fruits.length); // 3
```

Searching Elements

we can search for elements inside array using one of the following methods

- `includes()` : Checks if an element exists in the array

```
[1, 2, 3].includes(2); // true
```

- `indexOf()` : Finds the **first occurrence** of an element

```
['a', 'b', 'a'].indexOf('a'); // 0
```

- `lastIndexOf()` : Finds the **last occurrence** of an element (searches from the end)

```
['a', 'b', 'a'].lastIndexOf('a'); // 2
```

- `find()` : Returns the **first element** matching a condition

```
let users = [{id: 1}, {id: 2}];  
users.find(user => user.id === 2); // {id: 2}
```

Adding and Remove Elements

1. `push()` : Add to End

```
let fruits = ['apple', 'banana'];  
fruits.push('orange');  
// ['apple', 'banana', 'orange']
```

2. `unshift()` : Add to Start

```
let fruits = ['apple', 'banana'];  
fruits.unshift('orange');  
// ['orange', 'apple', 'banana']
```

3. `pop()` : Remove Last Element

```
let fruits = ['apple', 'banana', 'orange'];  
const last = fruits.pop();  
// fruits = ['apple', 'banana']  
// last = 'orange'
```

4. `shift()` : Remove First Element

```
let fruits = ['apple', 'banana', 'orange'];  
const first = fruits.shift();
```

```
// fruits = ['banana', 'orange']  
// first = 'apple'
```

5. `splice()` : Performs three key operations on arrays **Adds elements** at any specified position, **Removes elements** from any position, **Replaces elements** by combining addition and removal

```
let colors = ['red', 'green', 'blue'];  
  
// 1. Add without removing (deleteCount = 0)  
colors.splice(1, 0, 'yellow');  
// ['red', 'yellow', 'green', 'blue']  
  
// 2. Remove without adding  
colors.splice(2, 1);  
// ['red', 'yellow', 'blue']  
  
// 3. Replace elements  
colors.splice(1, 1, 'purple');  
// ['red', 'purple', 'blue']
```

- **First Argument (Required)** The index position where the operation should be performed
- **Second Argument (Required)** The number of existing elements to remove
- **Third Argument (Optional)** New element(s) to insert at the specified position

Combining, Spreading and Slicing

1. `concat()` concatenates and merges two or more arrays into a single new array

```
let arr1 = [1, 2];  
let arr2 = [3, 4];  
let combined = arr1.concat(arr2); // [1, 2, 3, 4]
```

2. `( ... )` The spread operator expands an array into its individual elements

```
// Combine arrays  
let merged = [...arr1, ...arr2]; // [1, 2, 3, 4]
```

```
// Insert additional elements
let withExtras = [...arr1, 'a', ...arr2, 'b']; // [1, 2, 'a', 3, 4, 'b']
```

3. `slice()` creates a shallow copy of a portion of an array without modifying the original array.

```
let original = [1, 2, 3];
let copy1 = original.slice(); // copy1 = [1, 2, 3]
let copy2 = original.slice(1); // copy2 = [2, 3]
```

- First argument is **start index** (inclusive)
- Second argument is **end index** (exclusive)
- Using 1 argument: extracts from start index to end
- Using no arguments: extracts entire array (full copy)

Sorting Array

- `sort()` It arranges array elements **strings alphabetically** and for number it require compare function

```
let fruits = ['banana', 'apple', 'orange'];
fruits.sort(); // ['apple', 'banana', 'orange']
[40, 1, 5, 200].sort((a, b) => a - b); // small to big
[40, 1, 5, 200].sort((a, b) => b - a); // big to small
```

- `reverse()` : It flips the current order of elements:

```
let letters = ['a', 'b', 'c'];
letters.reverse(); // ['c', 'b', 'a']
```

Objects:

Same as array we can do more then just creating and reading objects,we can

- **Add** or **remove** properties
- **Search** for specific values
- **Transform** object structure
- **Combine** or **split** objects
- **Freeze/seal** objects for protection

Getting Values And keys

- `Object.keys()` : Get all keys inside our object

```
let person = {"name": 'Alex', "age": 30};  
Object.keys(person); // ["name", "age"]
```

- `Object.values()` : Get all values inside object

```
let person = {"name": 'Alex', "age": 30};  
Object.values(person); // ["Alex", 30]
```

Object Length

When we retrieve an object's keys using `Object.keys()` , it returns an **array of property names**. We can then use `.length` on this array to determine the object's property count

```
let person = {"name": 'Alex', "age": 30};  
Object.keys(person).length; // 2
```

Searching Object

- `hasOwnProperty()` : Checks if an object **directly contains** a specified property

```
let person = {"name": 'Alex', "age": 30};  
person.hasOwnProperty('age'); // true  
person.hasOwnProperty('id'); // false
```

- `in` Checks if a property exists **anywhere** in the object

```
let person = {"name": 'Alex', "age": 30};  
"name" in person; // true
```

- `Object.values()` + `includes()` Check if value exist in the object

```
let person = {"name": 'Alex', "age": 30};  
Object.values(person).includes('Alex'); // true
```


Adding and Removing Properties

- We can add or modify object properties using dot or bracket notation

```
const user = {};  
user.name = 'Alex';    // Adds new property  
user.name = 'Sarah';   // Modifies existing property  
user["age"] = 20;
```

- We can remove properties from objects using the `delete` keyword

```
let person = {"name": 'Alex', "age": 30};  
delete person["age"]
```

Combining and Spreading Object

- We can combine multiple objects into a single object using `Object.assign()`

```
let obj1 = { a: 1 };  
let obj2 = { b: 2 };  
let merged = Object.assign({}, obj1, obj2); // { a: 1, b: 2 }
```

- We can use the spread operator (`...`) to work with objects

```
let obj1 = { a: 1 };  
let obj2 = { b: 2 };  
let merged = {...obj1, ...obj2}; // { a: 1, b: 2 }
```

Freeze and Seal

These methods help lock objects to prevent changes:

- `Object.freeze()`

Prevents all modifications:

- Cannot change existing values
- Cannot add new properties
- Cannot remove properties

```
let person = {"name": 'Alex', "age": 30};
Object.freeze(person);
person.name = 'Max'; // Fails silently
```

- **Object.seal()**

Allows only value changes:

- Can modify existing values
- Cannot add new properties
- Cannot remove properties

```
let person = {"name": 'Alex', "age": 30};
Object.seal(person);
person.id = '49416548'; // Fails silently
```

Comparison and Logical Operators

Comparison operators

Comparison operators help us compare values or variables with each other and create conditions based on the results. These comparisons always return a boolean value either `true` or `false`.

JavaScript provides us with the following comparison operators:

- **> (Greater than)**: Checks if the first operand is greater than the second.

```
5 > 4    // true
3 > 10   // false
```

- **< (Less than)**: Checks if the first operand is less than the second.

```
4 < 5    // true
7 < 3    // false
```

- **== (Equal to)**: Checks if both operands are equal.

```
4 == 4    // true
"hi" == "hello" // false
```

- **!= (Not equal to)**: Checks if the operands are not equal.

```
1 != 0    // true
5 != 5    // false
```

- **===**: Checks **strict equality** both value **and** type must match.

```
let a = 4;
let b = "4";
console.log(a === b); // false
```

- **!==**: Checks **strict non-equality** true if value **or** type differ.

```
let a = 4;
let b = "4";
console.log(a !== b); // true
```

- **>= (Greater than or equal to)**: Checks if the first operand is greater than or equal to the second.

```
2 >= 2    // true
3 >= 5    // false
```

- **<= (Less than or equal to)**: Checks if the first operand is less than or equal to the second.

```
2 <= 3    // true
4 <= 1    // false
```

Logical Operators

JavaScript provides logical operators to combine multiple conditions and build more complex expressions. The result of a logical operation is always a boolean value either true or false.

|| (OR)

- Returns **true** if **at least one** of the conditions is true.
- Returns **false** only if **all** the conditions are false.

```
console.log(true || false)    // true
console.log(false || false)   // false
```

&& (AND)

- Returns `true` only if **all** the conditions are true.
- Returns `false` if **any** of the conditions is false.

```
console.log(true && true)      // true
console.log(true && false)     // false
```

! (NOT)

- Reverses the logical state of the condition.
- If the condition is `true`, `!` makes it `false`, and vice versa.

```
console.log(!true)           // false
console.log(!false)          // true
```

Truth Table for Logical Operators

A	B	A && B	A B	!A
true	true	true	true	false
true	false	false	true	false
false	true	false	true	true
false	false	false	false	true

Conditional Statements

Conditional statements are fundamental programming constructs that enable our code to make decisions and execute different actions based on specific conditions. They control program flow by evaluating whether certain criteria are met, allowing for dynamic behavior that responds to different situations, inputs, or values.

`if`, `else`, and `else if` Statement

The first conditional statement in JavaScript is the `if...else` **statement**. It allows us to execute code based on conditions. There are three basic ways to use it:

Single Condition

This block runs **only** when the condition evaluates to `true`:

```
if (condition) {  
  // Code to run if condition is true  
}
```

Condition with Alternative

- Run one block if true, another if false:

```
if (condition) {  
  // Code to run if true  
} else {  
  // Code to run if false  
}
```

Multiple Conditions

- Chain conditions with `else if` for alternative paths:

```
if (condition1) {  
  // Code for condition1  
} else if (condition2) {  
  // Code for condition2  
} else {  
  // Default code if all conditions fail  
}
```

Switch Case Statements

The `switch` statement allows **us** to execute different code blocks based on the value of an expression. It's ideal for scenarios with **multiple fixed conditions** (e.g., menu options, status codes).

```
switch (expression) {
  case value1:
    // Code for value1
    break;
  case value2:
    // Code for value2
    break;
  default:
    // Code if no case matches
}
```

Example

```
let day = "Monday";
switch (day) {
  case "Monday":
    console.log("Weekday 1");
    break;
  case "Tuesday":
    console.log("Weekday 2");
    break;
  default:
    console.log("Not Monday/Tuesday");
}
// It will display "Weekday 1"
```

Loops

Loops allow us to repeat a block of code multiple times either a set number of times or until a certain condition is met. This helps us avoid repetition and makes our programs more efficient.

while Loop

We use a `while` loop when we want to repeat something as long as a condition is true.

```
void main(){
  int i = 0;

  while (i < 3) {
    print('i is $i');
  }
```

```
i++;  
}  
}
```

This loop keeps running while `i` is less than 3.

do-while Loop

A `do-while` loop is like a `while` loop, but it **runs at least once**, because the condition is checked **after** the first run.

```
void main(){  
    int i = 0;  
  
    do {  
        print('i is $i');  
        i++;  
    } while (i < 2);  
}
```

for Loop

We use a `for` loop when we know how many times we want to repeat something.

```
void main(){  
    for (int i = 0; i < 5; i++) {  
        print('Count: $i');  
    }  
}
```

This prints numbers from 0 to 4.

`i++` increases the counter each time.

for-in Loop

We use a `for-in` loop to go through each element in a collection like a list.

```
void main(){  
    List<String> fruits = ['apple', 'banana', 'cherry'];
```

```
for (var fruit in fruits) {  
    print(fruit);  
}  
}
```

This prints each fruit in the list.

break and continue

- `break` : exits the loop completely when condition is met.
- `continue` : skips to the next iteration when condition is met

Example 1

```
void main(){  
for (int i = 0; i < 5; i++) {  
    if (i == 3) break;  
    print(i); // Stops printing at 3  
}  
}
```

Example 2

```
void main(){  
for (int i = 0; i < 5; i++) {  
    if (i == 2) continue;  
    print(i); // Skips printing 2  
}  
}
```

Tasks

Task 1

You have the following array:

```
let fruits = ["apple", "banana", "mango"];
```

- Add `"orange"` to the **start** of the array.

- Replace "banana" with "kiwi" using `indexOf()` and direct assignment.
- Use `.join(", ")` to turn the array into a single string.

Task 2

You have the following object:

```
let user = {  
  username: "skywalker",  
  hobbies: ["coding", "gaming", "reading"]  
};
```

- Add a new hobby "traveling" using `.push()`.
- Convert all hobbies to **uppercase**.
- Create a summary like: "SKYWALKER enjoys CODING, GAMING, READING, TRAVELING."

Task 3

Create a script that simulates a decision-making process based on weather conditions. The script should:

- Declare variables `isRaining` and `isWarm` (both Boolean values).
- Use a combination of **logical operators** to decide whether you need to bring an umbrella and wear a jacket:
 - If it is raining and not warm, print "Bring an umbrella and wear a jacket."
 - If it is raining but warm, print "Bring an umbrella but no jacket needed."
 - If it is not raining but warm, print "No umbrella needed, but wear light clothes."
 - If it is neither raining nor warm, print "No umbrella and no jacket needed."

Task 4

Write a script that prints numbers from 1 to 100, but with a twist:

- For numbers divisible by 3, print "Fizz".
- For numbers divisible by 5, print "Buzz".
- For numbers divisible by both 3 and 5, print "FizzBuzz".
- For all other numbers, print the number itself.